

CANDIDATE ASSESSMENT

Demo Project Brief

Full-Stack Developer — Technical Assignment

Submission: [GitHub Repo](#) + [Video Walkthrough](#)

□ Read This Before You Start

This is a real engineering test. There is no right answer you can Google. The task is deliberately complex to simulate the kind of work you will do on Day 1. We are evaluating your thinking, your decisions, and your code quality — not just whether it compiles.

AI-generated code will be disqualified. We use automated similarity detection and code pattern analysis. If your code reads like it was written by a language model, it will be rejected without discussion. Write it yourself.

□ The Project — NexusDial

You will build NexusDial: a lightweight, multi-tenant virtual phone number SaaS backend with an AI-powered contact intelligence layer and a React Native mobile interface.

This is not a tutorial project. It touches telephony, real-time systems, multi-tenant security, AI pipelines, and mobile development simultaneously. Scope it well. Build what you can, document what you can't.

Why this project?

It deliberately mirrors the architecture of the products you'll inherit. If you can build NexusDial, you can navigate our codebase on Day 1. That's the test.

□ Module 1 — Backend API (Node.js + TypeScript)

1.1 Project Setup

- Node.js 20+ with TypeScript (strict mode on — no ts-ignore, no any)
- Express.js as the HTTP framework
- PostgreSQL as the database (Docker Compose setup required so we can run it locally)
- Prisma as the ORM — schema-first, all migrations committed
- Redis for session state, rate limiting, and job queuing
- Bull or BullMQ for async job processing
- Winston for structured logging (JSON format — no console.log in production code)
- Jest for testing — unit tests and at least one integration test per major flow

1.2 Database Schema — Build Exactly This

Design and implement a multi-tenant schema with the following models. Every table must include a `tenantId` (or `merchantId`) field. Every query on tenant-scoped data must filter by this field — we will check.

Tenant	<code>id</code> , <code>businessName</code> , <code>mobile</code> (unique), <code>planTier</code> (BASIC/PRO), <code>createdAt</code>
VirtualNumber	<code>id</code> , <code>tenantId</code> (FK), <code>e164Number</code> (unique), <code>label</code> , <code>isActive</code> , <code>provisionedAt</code>
Contact	<code>id</code> , <code>tenantId</code> (FK), <code>phoneNumber</code> , <code>name</code> (nullable), <code>tags</code> (string[]), <code>firstSeenAt</code> , <code>callCount</code>
CallRecord	<code>id</code> , <code>tenantId</code> (FK), <code>virtualNumberId</code> (FK), <code>contactId</code> (FK nullable), <code>direction</code> , <code>status</code> , <code>durationSec</code> , <code>recordingKey</code> , <code>aiSummary</code> (nullable), <code>createdAt</code>
IntelligenceJob	<code>id</code> , <code>callRecordId</code> (FK unique), <code>status</code> (PENDING/PROCESSING/DONE/FAILED), <code>transcript</code> , <code>extractedData</code> (JSON), <code>processingMs</code> , <code>createdAt</code>
WalletTx	<code>id</code> , <code>tenantId</code> (FK), <code>type</code> (CREDIT/DEBIT), <code>amountPaise</code> (INT — no floats), <code>description</code> , <code>createdAt</code>

1.3 Authentication

OTP-based authentication only. No passwords. No social login.

- POST `/api/auth/send-otp` — accepts mobile number, generates 6-digit OTP, stores in Redis with 5-min TTL, 'sends' it (just log it to console — no real SMS needed for the demo)
- POST `/api/auth/verify-otp` — validates OTP, returns JWT access token (15-min expiry) + refresh token (30 days, stored in DB)
- POST `/api/auth/refresh` — rotates refresh token, returns new access token
- Rate limit: max 3 OTP requests per mobile per 10 minutes
- All protected routes use a `validateToken` middleware that attaches `req.tenant`

1.4 Virtual Number Management

- GET `/api/numbers` — list tenant's virtual numbers
- POST `/api/numbers` — provision a new number (pick from a `NumberPool` table — same concept as `DIDPool`). Auto-assign and mark as taken.
- PATCH `/api/numbers/:id` — update label, toggle `isActive`
- When `NumberPool` drops below 5 unassigned numbers, log a structured WARNING — pretend it's a Slack alert

1.5 Call Simulation Endpoint

Since you're not connecting to a real PBX, simulate call events via an API endpoint:

- POST `/api/simulate/call` — accepts { `virtualNumberId`, `callerMobile`, `direction`, `durationSec`, `hasVoicemail` }
- This endpoint creates a `CallRecord` with status `ANSWERED` (or `MISSED` if `durationSec` is 0)
- If `hasVoicemail` is true, push a job to the Bull queue to process the 'voicemail'
- Emit a Socket.io event to the tenant's room: { event: 'call_event', data: `callRecord` }

- Return the created CallRecord in the response

1.6 AI Contact Intelligence Pipeline (The Hard Part)

This is the module we'll scrutinise most carefully. It must be async, cost-controlled, and fault-tolerant.

When a voicemail job is processed by the Bull worker:

- Step 1 — Transcription (Simulated): Generate a fake transcript using a simple randomised script. Pick from a set of 5-10 hardcoded realistic Hindi/English voicemail texts. Do NOT use a real STT API — this is intentional. We want to see how you handle the interface, not the API cost.
- Step 2 — AI Extraction: Call the Gemini API (gemini-1.5-flash) OR the OpenAI API (gpt-3.5-turbo) — your choice. Pass the transcript. Prompt the model to extract: { name: string|null, intent: string (max 20 words)|null, sentiment: 'positive'|'neutral'|'negative', callbackRequested: boolean }. Validate the response with Zod before trusting it.
- Step 3 — Contact Intelligence: Look up the Contact record by callerMobile + tenantId. If it exists, increment callCount. If it doesn't exist, create it with extractedName (if found). Update the Contact.tags array with the extracted intent as a tag if it's new.
- Step 4 — Persist: Update IntelligenceJob to DONE with transcript + extractedData. Update CallRecord.aiSummary with a one-line summary.
- Step 5 — Notify: Emit a Socket.io event 'intelligence_ready' to the tenant's room with the enriched CallRecord.

Failure Handling Requirements

If the AI API call fails: mark the job FAILED, log the error, do NOT retry more than 2 times. The transcript must still be saved even if AI extraction fails — never lose the raw data. Failed jobs must be inspectable (not auto-deleted).

1.7 Contacts API

- GET /api/contacts — paginated list with filters: tag, callCount (min), firstSeenAt (date range)
- GET /api/contacts/:id — full contact detail including last 5 call records
- PATCH /api/contacts/:id — update name, add/remove tags
- GET /api/contacts/:id/timeline — chronological list of all CallRecords for this contact
- DELETE /api/contacts/:id — soft delete (mark isDeleted, keep data for 30 days)

1.8 Analytics Endpoint

One endpoint that an MSME dashboard would actually use:

- GET /api/analytics/summary — returns: totalCallsToday, missedCallsToday, newContactsThisWeek, topCallers (top 5 contacts by callCount), sentimentBreakdown ({ positive: N, neutral: N, negative: N } from IntelligenceJob.extractedData)
- All data scoped to req.tenant.id — any cross-tenant data leak is an automatic disqualification

□ Module 2 — React Native App

A clean, functional mobile app that connects to your backend. Design is secondary to functionality and code quality — but empty white screens are not acceptable.

2.1 Required Screens

Login Screen	Mobile number input → OTP input → JWT stored in Keychain/Keystore (NOT AsyncStorage). This is non-negotiable — we will check.
Dashboard	Today's call stats (total, missed, new contacts). Socket.io connected — incoming 'call_event' shows a non-blocking toast with caller number. Pull-to-refresh.
Call Log	Paginated FlatList. Direction icon, caller, virtual number label, duration or MISSED badge, AI summary snippet if available. Tap to open Call Detail.
Call Detail	Full call metadata. If IntelligenceJob is DONE: show extracted name, intent, sentiment badge, callbackRequested flag. If PENDING: show skeleton loader with 'Processing...' label.
Contacts	Searchable list. Each card: name or phone, tags (rendered as chips), call count. Tap for Contact Timeline.
Contact Timeline	Chronological feed of all calls from this contact with AI summaries. Looks like a messaging thread but for calls.
Simulate Call	A debug screen (accessible from Settings). Form: virtual number selector, caller mobile input, direction toggle, duration slider, voicemail toggle. Submit calls POST /api/simulate/call.

2.2 Technical Requirements

- TypeScript throughout — no implicit any
- React Query (TanStack Query) for all API calls — no raw useEffect fetching
- Zustand or Redux Toolkit for global state (auth token, active call)
- Socket.io client connected on app launch, reconnects automatically
- react-native-keychain for secure token storage — this will be checked
- React Navigation with at least a bottom tab navigator + stack navigators per tab
- Every list must have: loading skeleton, empty state with message, error state with retry button
- No hardcoded colours — use a theme/constants file

❑ Module 3 — Security & Multi-Tenancy (This Gets You the Job)

This module has no code to write — it's about how you wrote Module 1. We will audit your code for these specific things:

Tenant isolation	Every single query on CallRecord, Contact, VirtualNumber, WalletTx, IntelligenceJob must have a WHERE tenantId = req.tenant.id clause. We will look for missing ones.
Zod validation	Every POST and PATCH request body has a Zod schema. If a field isn't in the schema, it doesn't reach the controller.
Rate limiting	At minimum: auth endpoints (3 OTP/10min/mobile), general API (200 req/15min/tenant). Use express-rate-limit with Redis store.
No secrets in code	All keys in .env. .env in .gitignore. A .env.example file committed with all required keys listed (values empty).
JWT storage	React Native app uses react-native-keychain. Never AsyncStorage. We will check the relevant source file.
Error codes	Consistent error response shape with machine-readable codes (e.g. ND_4001, ND_4029). No stack traces in API responses.
Input sanitation	Parameterised queries via Prisma (already enforced), but also: phone number format validation, string length limits on all text fields.

□ Module 4 — Testing

Write tests. Not many — but real ones.

- Unit test the AI extraction service — mock the Gemini/OpenAI client, test with 3 transcript variations: name stated, name not stated, empty transcript
- Unit test the tenant isolation check — a helper function that verifies a resource belongs to a tenant should be tested with both matching and mismatched tenant IDs
- Integration test the simulate-call endpoint: POST → verify CallRecord created → verify Bull job enqueued → verify Socket.io event emitted (mock the socket)
- Integration test one auth flow: send-otp → verify-otp → JWT issued → protected route accessible
- All tests must pass with npm test. CI badge in README is a bonus.

□ Submission Requirements

GitHub Repository Must Contain:

- README.md — clear setup instructions so we can run it locally in under 10 minutes. Include Docker Compose for PostgreSQL + Redis.
- docker-compose.yml — PostgreSQL + Redis, correct ports, volume mounts
- .env.example — all required environment variables listed (empty values)
- Prisma schema + all migrations committed
- src/ folder following a clear, logical structure
- test/ folder with passing tests

- Commit history that tells a story — not a single 'initial commit' with 3000 files

Video Walkthrough (5–10 minutes):

- Architecture overview — what you built and why you made the structural decisions you did
- Live demo: simulate a call (with voicemail), show the Socket.io event arrive in the app in real time, show the AI processing complete and appear in the Call Detail screen
- Code walkthrough: show the multi-tenant query pattern you used, show the Bull worker, show the Zod validation on one endpoint
- What you would do differently / what you'd build next if given another week
- Talk about a technical decision where you had two options and explain why you chose what you chose

Timeline:

Time Limit	No hard deadline — quality over speed. Most strong candidates take 5–8 days.
Partial Submissions	Submit what you have. A clean, well-structured Module 1 with honest documentation of what's missing is better than a broken Module 3.
Questions	Reach out if something in this brief is ambiguous. We prefer you ask than assume.

☐ How We Score It

Area	Weight	Max Points
Multi-tenant security — not a single cross-tenant leak	High	25
AI pipeline — fault tolerance, async correctness, cost-awareness	High	20
Code quality — TypeScript strictness, no any, clean structure	High	20
React Native — secure storage, real-time events, UX quality	Medium	15
Testing — real tests, not trivial snapshots	Medium	10
Video walkthrough — clarity of thought and architectural reasoning	Medium	10

Build something you're proud of.

The best submissions show clear thinking, honest trade-offs, and code that reads like the developer cares.

We're excited to see what you build. ⚡